

Ros Day02 hw2 AI 사용 보고서

작성자: Doyeon Park

작성일: 2026.09.10

목차

1. AI 보고서 개요
2. AI 사용 부분
3. 깨달은 점

1. AI 보고서 개요

ROS 2 Day2 과제 2 를 진행하면서 과제 요구사항 분석, ROS 2 패키지 구성, Qt GUI 코드 작성, 빌드 오류 해결, 도형이 잘못 그려지는 문제 분석에 AI 를 사용하였다. 주로 ChatGPT 와 Codex 를 이용하여 필요한 명령어와 코드 구조를 확인하였다.

AI 가 안내받은 내용을 직접 파일에 옮겨 적은 뒤 빌드와 실행을 반복하였다. 오류가 발생했을 때는 터미널 출력과 실행 화면을 AI 에 전달하고 원인을 분석하는 데 도움을 받았다.

최종 결과는 직접 turtlesim 과 GUI 를 실행하여 버튼, 키보드 입력, 도형 출력, cmd_vel 값 표시가 정상적으로 동작하는지 확인하였다.

2. AI 사용 부분

AI 는 먼저 ROS 2 C++ 패키지 생성과 Qt 설정에 사용하였다. rclcpp, geometry_msgs, turtlesim, std_srvs, Qt5 Widgets 의존성을 package.xml 과 CMakeLists.txt 에 추가하는 방법을 확인하였다. Qt 의 버튼과 슬롯을 사용하기 위해 Q_OBJECT 와 CMAKE_AUTOMOC 설정이 필요하다는 것도 AI 를 통해 확인하였다.

QShortcut 을 이용하여 GUI 버튼과 W, A, S, D, Space 키가 같은 기능을 실행하도록 구성하였다.

ROS 2 통신 부분에서는 /turtle1/cmd_vel 토픽으로 Twist 메시지를 전송하는 Publisher 를 작성할 때 AI 를 사용하였다. linear.x 에는 거북이의 선속도를 저장하고 angular.z 에는 회전 속도를 저장하였다. 전송하는 cmd_vel 값을 GUI 에서 확인할 수 있도록 linear.x 와 angular.z 를 QLabel 에 표시하였다.

거북이의 현재 위치를 확인하기 위해 /turtle1/pose 토픽을 구독하는 Subscriber 를 작성하였다. 수신한 x, y, theta 값은 GUI 에 표시하고 거북이의 이동 거리와 회전 각도를 계산할 때 사용하였다.

/turtle1/set_pen 서비스는 도형의 색상과 굵기를 변경할 때 사용하였다. /turtle1/teleport_absolute 서비스는 거북이를 각 도형의 시작 위치로 이동시킬 때 사용하였다. /clear 서비스는 화면에 그려진 선을 지울 때 사용하였다.

삼각형은 직진과 120 도 회전을 세 번 반복하도록 작성하였다. 사각형은 직진과 90 도 회전을 네 번 반복하도록 작성하였다. 원은 linear.x 와 angular.z 를 동시에 전송하여 곡선으로 이동하도록 작성하였다. 각 도형은 서로 다른 위치에서 시작하며 색상과 굵기도 다르게 설정하였다.

GUI 가 실행된 후에는 도형의 일부가 끊기거나 이상한 방향으로 그려지는 문제가 발생하였다. 실행 화면과 코드를 AI 에 도움을 받아 확인한 결과 teleport 직후 이전 pose 값을 기준으로 이동을 시작하는 것이 원인이었다.

이를 해결하기 위해 teleport 후 거북이의 pose 가 목표 위치와 방향에 도달했는지 확인한 다음 이동 명령을 실행하도록 수정하였다. 또한 도형을 그리는 중에는 다른 도형 명령이 중복으로 실행되지 않도록 처리하였다.

헤더도 정확해. 저장하고 종료한 다음 GUI 화면을 구현하자.

○ 채팅에 추가

Ctrl+O → Enter → Ctrl+X

</> Bash

nano src/turtle_gui.cpp

아래 코드를 입력해줘.

</> C++

```
#include "parkdoyeon_day2_hw2/turtle_gui.hpp"

#include <QGridLayout>
#include <QGroupBox>
#include <QLabel>
#include <QPushButton>
#include <QVBoxLayout>
#include <QWidget>

TurtleGui::TurtleGui(
    const std::shared_ptr<rcclcpp::Node> &node,
    QWidget *parent)
    : QMainWindow(parent),
      node_(node)
{
    createUi();
}

void TurtleGui::createUi()
{
    setWindowTitle("ROS 2 Turtlesim Controller");
    resize(600, 400);

    auto *central_widget = new QWidget(this);
    auto *main_layout = new QVBoxLayout(central_widget);

    auto *title_label =
        new QLabel("Turtlesim Shape Controller");

    title_label->setStyleSheet(
        "font-size: 24px; font-weight: bold;");

    main_layout->addWidget(title_label);

    auto *button_group =
        new QGroupBox("Shape Buttons");

    auto *button_layout =
        new QGridLayout(button_group);

    auto *triangle_button =
        new QPushButton("W - Triangle");

    auto *square_button =
        new QPushButton("A - Square");

    auto *circle_button =
        new QPushButton("S - Circle");
```

오른쪽 터미널의

○ 채팅에 추가



Oh no! I hit the wall!
Clamping from [x=-0.011...]

은 거북이가 화면 왼쪽 경계 밖으로 이동하려 했다는 뜻이야. 빨간 삼각형과 초록 사각형이 열린 상태인 걸 보면, 도형이 끝나기 전에 다른 버튼을 눌러 이전 그리기가 중단된 것으로 보여.

현재 코드는 버튼을 새로 누르면 진행 중인 도형을 취소하고 새 도형을 시작해. 빠르게 W→A→S→D를 누르면 순간이동 요청까지 겹쳐서 이런 현상이 생길 수 있어.

우선 두 프로그램을 종료하고 다시 실행해줘.

</> Bash



```
ros2 run turtlesim turtlesim_node
```

</> Bash



```
ros2 run parkdoyeon_day2_hw2 turtle_gui
```

이번에는 **D 전체**를 한 번만 누르고 상태가 **도형 그리기 완료**로 바뀔 때까지 기다려봐. 그래도 벽에 부딪히면 버튼 문제가 아니라 순간이동 직후 이전 pose를 사용하는 코드 문제이므로 그 부분을 고치면 돼.



?

새 터미널에서 ROS 작업공간을 **source** 하지 않아서 패키지를 못 찾는 거야.

GUI 실행 터미널에서:

</> Bash



```
source /opt/ros/jazzy/setup.bash  
source ~/colcon_ws/install/setup.bash
```

```
ros2 run parkdoyeon_day2_hw2 turtle_gui
```

새 터미널을 열 때마다 **source ~/colcon_ws/install/setup.bash**를 다시 실행해야 해당 작업공간의 패키지를 찾을 수 있어.

내 직접 반영 요청이 거절된 상태라 실제 colcon_ws 에는 아직 수정되지 않았어. VS Code 터미널에서 아래 명령으로 세 파일을 교체해줘.

```
</> Bash
cd ~/colcon_ws/src/parkdoyeon_day2_hw2

cp /home/doyeon/.codex/.chatgpt-projects/g-p-6a9fdff102a48191a22bd0a66eb4c04d/parkdoyeon_day2_hw2/include/parkdoyeon_day2_hw2/turtle_gui.hpp \
include/parkdoyeon_day2_hw2/turtle_gui.hpp

cp /home/doyeon/.codex/.chatgpt-projects/g-p-6a9fdff102a48191a22bd0a66eb4c04d/parkdoyeon_day2_hw2/src/turtle_gui.cpp \
src/turtle_gui.cpp

cp /home/doyeon/.codex/.chatgpt-projects/g-p-6a9fdff102a48191a22bd0a66eb4c04d/parkdoyeon_day2_hw2/src/main.cpp \
src/main.cpp

cd ~/colcon_ws
colcon build --packages-select parkdoyeon_day2_hw2
```

그다음 turtlesim과 GUI를 새로 실행해서 D 전체 를 테스트하면 돼.

📄 🗨️ ↩️ (수요일) 오후 10:02

잘 됨

좋아, 과제 2 구현 완료야.

영상 촬영 시 한 화면에 다음이 보이게 하면 돼.

- turtlesim과 Qt GUI
- W/A/S/D 키 및 버튼 동작
- 삼각형·사각형·원 완성
- 도형별 색상과 굵기 차이
- GUI의 linear.x , angular.z 값 변화
- 정지와 화면 지우기

제출 문서상 AI를 사용했으므로 별도의 AI 사용 보고서도 필요↓만 잊지 마.

3. 깨달은 점

이번 과제를 통해 Publisher, Subscriber, Service Client 의 차이를 알 수 있었다. Publisher 는 /turtle1/cmd_vel 로 속도 명령을 전송하고 Subscriber 는 /turtle1/pose 에서 현재 위치를 받는다. Service Client 는 펜 설정, 순간 이동, 화면 지우기처럼 한 번의 요청으로 실행되는 기능에 사용된다.

cmd_vel 의 linear.x 와 angular.z 가 거북이의 움직임에 어떤 영향을 주는지도 확인하였다. linear.x 값은 직진 속도를 결정하고 angular.z 값은 회전 속도를 결정한다. 두 값을 동시에 사용하면 원과 같은 곡선을 그릴 수 있다.

Qt 와 ROS 2 를 하나의 프로그램에서 함께 사용할 때는 두 프로그램의 이벤트를 모두 처리해야 한다는 점을 알게 되었다.
QTimer 를 이용하여 일정한 주기로 rclcpp::spin_some 을 실행하면 GUI 를 유지하면서 ROS 2 메시지와 서비스 응답을 처리할 수 있다는 것을 알게 되었다.

도형을 정확하게 그리기 위해서는 속도를 일정 시간 동안 전송하는 것보다 pose 값을 이용하여 실제 이동 거리와 회전 각도를 확인하는 방식이 더 정확하다는 것을 알게 되었다.